

AUTH SYSTEM — MODULE 1

Database & ORM Decision Document

PART 2 OF 3

Why PostgreSQL over MySQL — Why SQLAlchemy over Raw SQL

Purpose of This Document

This document answers two specific technical questions raised during the project review:

- Why PostgreSQL was chosen over MySQL, given that MySQL is also free and widely used
- Why SQLAlchemy ORM was chosen over writing raw SQL queries, given that ORM becomes complex at scale

Both questions are answered with complete honesty — including acknowledging the real weaknesses of each choice and explaining exactly when you should abandon the chosen approach.

Multi-Module Context

This is Module 1 of a larger project. Every database decision is evaluated not just for today but for the impact it will have when Module 2, 3, and beyond are added. This context changes some answers significantly.

SECTION A — Why PostgreSQL Instead of MySQL

MySQL is free, battle-tested, and runs half the internet. PostgreSQL is also free, also battle-tested, and runs the other half. So why choose one over the other? The answer depends entirely on what the application needs to do — both now and in the future.

A1. The Short Answer

One Sentence Answer

PostgreSQL was chosen because it enforces data integrity more strictly, handles concurrent users more safely, and supports advanced data types that this project and future modules will need — all without sacrificing performance or cost.

A2. Feature Comparison — Every Relevant Difference

Feature	PostgreSQL	MySQL
Cost	Free and open source	Free community edition
ACID compliance	Full — every transaction	Full since InnoDB engine
Default transaction isolation	Read Committed	Repeatable Read
Concurrent write handling	MVCC — readers never block writers	MVCC — similar but older implementation
JSON support	JSONB — binary, indexed, queryable	JSON — text stored, limited indexing
Full text search	Built in, powerful	Basic, limited
Array columns	Native array type	Not supported
Partial indexes	Supported	Not supported
Concurrent index creation	Yes — no table lock	No — locks table
CHECK constraints	Fully enforced	Parsed but NOT enforced (ignored)
Foreign key enforcement	Always enforced by default	Requires InnoDB, can be disabled
Window functions	Full support	Full support since 8.0
CTEs (WITH clause)	Full including recursive	Full since 8.0
Stored procedures	PL/pgSQL — very powerful	Supported but limited
Triggers	Full support	Full support
Replication	Streaming, logical, both	Binary log based
asyncpg driver (Python)	Excellent — built for	aiomysql — functional but less

Feature	PostgreSQL	MySQL
async)	PostgreSQL	mature
Cloud managed options	AWS RDS, Aurora, Google Cloud SQL, Supabase	AWS RDS, Aurora MySQL, PlanetScale
License	PostgreSQL License — truly free	GPL — some commercial restrictions

A3. The Five Reasons That Actually Matter for This Project

Reason 1 — CHECK Constraints Are Actually Enforced

This is one of the most surprising differences and one of the most important for data integrity.

```
-- This constraint in MySQL is SILENTLY IGNORED
CREATE TABLE users (
  age INT CHECK (age >= 0 AND age <= 150)
);

-- MySQL accepts this without error:
INSERT INTO users (age) VALUES (-50);
-- Row inserted. Constraint did nothing.

-- The same constraint in PostgreSQL:
INSERT INTO users (age) VALUES (-50);
-- ERROR: new row violates check constraint
-- Data protected at the database level
```

For an authentication system, this matters for fields like OTP attempt counts, token expiry validation, and status flags. PostgreSQL guarantees the database itself enforces these rules — not just the application code. If a bug in the application tries to write invalid data, the database rejects it.

Reason 2 — JSONB for Future Module Flexibility

This project is Module 1. Future modules will store configuration data, feature flags, user preferences, audit details, and integration settings that do not fit neatly into fixed columns.

```
-- PostgreSQL JSONB — stored as binary, fully indexed
SELECT * FROM audit_logs
WHERE detail->>'event_type' = 'login'
AND detail->>'country' = 'India';

-- You can index JSONB fields:
CREATE INDEX idx_audit_detail_event
ON audit_logs USING gin (detail);

-- MySQL JSON — stored as text
-- Cannot be indexed the same way
-- Queries are slower on large datasets
```

The `audit_logs` table already stores a JSON detail column. In PostgreSQL this can be queried efficiently. In MySQL it cannot be indexed and querying it requires a full table scan.

Reason 3 — Partial Indexes for Performance at Scale

The code review correctly identified that a partial index is needed on `user_sessions` for active sessions only. This feature does not exist in MySQL.

```
-- PostgreSQL partial index — only indexes active sessions
-- Index stays small even with millions of old sessions
CREATE INDEX idx_sessions_active
ON user_sessions (user_id)
WHERE is_valid = true;

-- MySQL cannot do this
-- Must index ALL sessions including invalidated ones
-- Index grows forever as sessions accumulate
-- Queries slow down over time
```

In a multi-module project where millions of sessions accumulate over time, this difference becomes significant. MySQL's index would grow unboundedly. PostgreSQL's partial index stays small and fast.

Reason 4 — Concurrent Index Creation Without Locking

When the application is live and needs a new index added, there are two very different experiences:

PostgreSQL	MySQL
CREATE INDEX CONCURRENTLY takes minutes on a large table. Application keeps running. Users are unaffected. Zero downtime.	CREATE INDEX locks the entire table. Application cannot write to the table during the operation. For a large table this means minutes of downtime.

For a growing multi-module project where new indexes will be needed regularly, this matters enormously. Scheduled maintenance windows become unnecessary with PostgreSQL.

Reason 5 — `asyncpg` Driver Quality

This project uses FastAPI with full `async/await`. The database driver quality directly affects performance under load.

Metric	<code>asyncpg</code> (PostgreSQL)	<code>aiomysql</code> (MySQL)
Protocol	Binary protocol — faster	Text protocol — slower
Performance	Fastest async Python DB driver available	Functional but significantly slower

Metric	asyncpg (PostgreSQL)	aiomysql (MySQL)
Maintenance	Actively maintained, well funded	Less active development
SQLAlchemy integration	First-class support	Good support

A4. When MySQL Would Have Been the Better Choice

Honesty requires acknowledging when the other choice is better. MySQL would have been preferred if:

- The team already had deep MySQL expertise and no PostgreSQL experience — familiarity reduces bugs
- The hosting provider only offered MySQL managed instances at acceptable cost
- The project required PlanetScale serverless MySQL, which has excellent developer experience
- The application was purely simple CRUD with no complex queries, JSON, or advanced indexing
- Budget required the cheapest possible managed database and MySQL was cheaper on that specific platform

None of these conditions apply to this project. PostgreSQL is the correct choice here.

SECTION B — Why SQLAlchemy Instead of Raw SQL

The reviewer raised a specific and important concern: ORM is not suitable for complex models as the project grows. This concern deserves a serious and honest response — not a dismissal.

B1. The Reviewer's Concern — Stated Fairly

The reviewer's argument is a well-known and legitimate criticism of ORM:

- ORM generates SQL behind the scenes that you cannot control
- As models grow complex, the generated SQL becomes inefficient
- Debugging ORM queries is harder than debugging raw SQL you wrote yourself
- N+1 query problems are easy to create accidentally with ORM relationships
- For complex reporting queries, ORM syntax becomes more confusing than the SQL it generates

Every one of these points is factually correct. They are real problems that real teams hit with ORM. The question is not whether these problems exist — they do — but whether they apply to this specific project and whether the trade-offs favour ORM or raw SQL.

B2. Why SQLAlchemy Was Chosen Despite These Concerns

Reason 1 — SQL Injection Prevention Is Automatic

The most important security requirement of this project is preventing SQL injection. With raw SQL, this requires discipline on every single query. With SQLAlchemy ORM, it is impossible to accidentally write an injectable query.

```
# RAW SQL — injection possible if you are not careful
# A tired developer writes this one night:
username = request_data['username']
query = f"SELECT * FROM users WHERE username = '{username}'"
# If username = "admin' OR '1'='1"
# The query becomes: SELECT * FROM users
# WHERE username = 'admin' OR '1'='1'
# This returns ALL users — security breach

# SQLAlchemy ORM — injection is structurally impossible
result = await db.execute(
    select(User).where(User.username == username)
)
# SQLAlchemy always uses parameterised queries
# The username value is NEVER interpolated into SQL text
```

```
# Even a malicious value is treated as data, not SQL
```

For an authentication module handling user credentials, this automatic protection is worth significant trade-offs in query complexity.

Reason 2 — Database Migration Management

SQLAlchemy works with Alembic to manage schema changes over time. This is essential for a multi-module project where the schema will evolve continuously.

```
# With SQLAlchemy models, Alembic can detect changes
# automatically and generate migrations:

# You add a new column to the User model:
profile_picture_url: Mapped[str | None] = mapped_column(
    String(500), nullable=True
)

# Alembic detects it and generates the migration:
# alembic revision --autogenerate -m 'add profile picture'

# Generated migration file:
def upgrade():
    op.add_column('users',
        sa.Column('profile_picture_url',
            sa.String(500), nullable=True))

# Applied with: alembic upgrade head
# Rolled back with: alembic downgrade -1
```

With raw SQL, every schema change requires manually writing migration files. There is no automatic detection. Missing a column in a migration causes production failures.

Reason 3 — Relationship Navigation Without Extra Queries

```
# With SQLAlchemy relationships defined in models,
# you can navigate related data naturally:

# Load user with all their active sessions
result = await db.execute(
    select(User)
    .options(selectinload(User.sessions))
    .where(User.id == user_id)
)
user = result.scalar_one()

# Access sessions without extra query
active_sessions = [s for s in user.sessions if s.is_valid]
```



```
# SQLAlchemy generates ONE efficient JOIN query
# Not N+1 queries (selectinload prevents this)
```

Reason 4 — Type Safety Across the Entire Stack

SQLAlchemy 2.x with `mapped_column` and `Mapped` types gives full Python type checking on every database operation. Your IDE catches errors before the code runs.

```
# SQLAlchemy 2.x mapped columns are fully typed
class User(Base):
    email: Mapped[str] = mapped_column(String(254))
    is_active: Mapped[bool] = mapped_column(Boolean)
    last_login: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )

# Your IDE knows that user.email is a str
# user.is_active is a bool
# user.last_login is datetime or None

# With raw SQL you get back a dict or Row object
# row['email'] — the IDE has no idea what type this is
# Bugs appear at runtime, not at development time
```

B3. The Reviewer Is Right That ORM Struggles — Here Is When

This is the most important part of this section. The reviewer's concern is valid and the correct answer is not to defend ORM blindly — it is to define exactly when ORM is appropriate and when it should be bypassed.

Query Type	Use ORM?	Why
Single record lookup by ID	Yes	Clean, safe, simple
Insert or update a record	Yes	Migration-safe, type-checked
Simple filtered list	Yes	Readable and maintainable
Multi-table JOIN with filters	Yes — SQLAlchemy Core	ORM join syntax is still clean
Complex aggregation report	No — use raw SQL	GROUP BY with HAVING is cleaner in SQL
Dashboard analytics query	No — use raw SQL	ORM makes this unreadable
Bulk insert of thousands of rows	No — use execute() with raw SQL	ORM row-by-row is too slow
Full text search query	No — use raw SQL	PostgreSQL FTS syntax is cleaner directly
Window functions	Partially — use text() clause	SQLAlchemy supports but syntax is verbose

B4. The Correct Approach — Hybrid Strategy

The professional answer is not ORM or raw SQL — it is both, used where each is appropriate. SQLAlchemy supports this natively through its text() function:

```
from sqlalchemy import text

# Use ORM for standard operations (most of the code)
user = await db.get(User, user_id)
user.last_login_at = datetime.now(timezone.utc)
await db.commit()

# Drop to raw SQL when the query is genuinely complex
# SQLAlchemy executes it safely with parameterisation
result = await db.execute(
    text(
        """
        SELECT
            u.id,
            u.email,
            COUNT(s.id) as session_count,
            MAX(s.created_at) as last_session
        FROM users u
```

```
LEFT JOIN user_sessions s
  ON s.user_id = u.id
  AND s.is_valid = true
WHERE u.created_at > :since
GROUP BY u.id, u.email
HAVING COUNT(s.id) > :min_sessions
ORDER BY last_session DESC
LIMIT :limit
'''
),
{
  'since': since_date,
  'min_sessions': 1,
  'limit': 100
}
)
rows = result.fetchall()

# Parameters are still bound safely
# SQL injection is still impossible
# You write clean readable SQL
# You get type-safe results back
```

This is the pattern used by large-scale production systems. Standard operations use ORM for safety and maintainability. Complex queries use raw SQL through SQLAlchemy's `text()` function for clarity and performance — but still with parameterised values, never string concatenation.

B5. What About Raw SQL Libraries Instead of SQLAlchemy?

Some projects use asyncpg directly without SQLAlchemy at all. Here is an honest comparison:

Concern	SQLAlchemy + asyncpg	Pure asyncpg (raw SQL)
SQL Injection	Impossible with ORM, safe with text()	Developer must always use \$1 parameters
Migrations	Alembic auto-detects model changes	Must write every migration manually
Type safety	Full type checking via Mapped[]	Manual type casting from Row objects
Complex queries	text() for full SQL control	Natural — write SQL directly
Performance	Slight overhead for ORM operations	Maximum performance — zero overhead
Learning curve	Higher — must learn ORM + SQL	Lower — just SQL
Multi-module scaling	Models shared across modules easily	SQL queries duplicated across modules
Code volume	Less code for standard operations	More boilerplate for every query

For a single module project written by one developer who is an expert in SQL, pure asyncpg is a reasonable choice. For a multi-module project where different developers will work on different modules, SQLAlchemy models serve as the shared contract — every module knows exactly what a User or UserSession looks like.

B6. The Plan for Complex Queries in Future Modules

As the project grows and queries become more complex, the hybrid strategy described above scales cleanly. Here is the recommended pattern for future modules:

```
# app/db/queries/user_queries.py
# All complex raw SQL lives in dedicated query files
# Separated from endpoint logic
# Easy to find, test, and optimise independently

from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession

async def get_user_activity_summary(
    db: AsyncSession,
    user_id: str,
    days: int = 30
) -> list[dict]:
    """
```

Complex query using raw SQL via text().

Parameters are bound safely — no injection risk.

```
'''
result = await db.execute(
    text("""
        SELECT date_trunc('day', created_at) as day,
               COUNT(*) as event_count,
               COUNT(DISTINCT ip_address) as unique_ips
        FROM audit_logs
        WHERE user_id = :user_id
        AND created_at > NOW() - INTERVAL :days DAY
        GROUP BY 1
        ORDER BY 1 DESC
    """),
    {'user_id': user_id, 'days': days}
)
return [dict(row) for row in result.fetchall()]
```

Separating complex queries into dedicated files means the ORM models stay clean, the complex SQL is easy to find and review, and performance-critical queries can be optimised without touching application logic.

Final Summary — Both Decisions

Decision	Choice Made	When to Reconsider
Database	PostgreSQL	Only if the team has strong MySQL expertise AND the hosting platform makes MySQL significantly cheaper
ORM Strategy	SQLAlchemy for standard operations, text() for complex queries	If more than 30% of queries become complex analytics, consider a dedicated read-replica with raw SQL query layer

The Reviewer's ORM Concern — Final Response

The reviewer is correct that ORM becomes problematic for complex queries. The solution is not to avoid ORM entirely — it is to use ORM where it excels (standard CRUD, migrations, type safety, injection prevention) and drop to parameterised raw SQL via text() where the query genuinely requires it. This hybrid approach is used by companies like Instagram, Dropbox, and Reddit — all of which use SQLAlchemy in production at massive scale.